

2. Post-Install Sensor-Only Test (No AI Modules)

Test # Action What to Measure Acceptable Delta vs. Baseline

2.1 TCP send iperf3 c <peer> -t 60 (sensor running) Gbps $\leq 5\%$ reduction from To

2.2 UDP send iperf3 c <peer> -u-b 10G -t 60 Gbps, loss% $\leq 10\%$ reduction from U.

2.3 TCP receive Same as 1.3 Gbps $\leq 5\%$ from Torx

2.4 Latency ping -c 100 Avg RTT, p99 ≤ 1 ms increase from Lo

2.5 I/O (storage) fio same params as 1.5 IOPS, MB/s, p99 latency $\leq 5\%$ from 10.

2.6 CPU mpstat during iperf run Per-core peak $\leq 25\%$ of one core (per CrowdStrike config default)

2.7 Memory free -m during iperf run RSS of falcon-sensor ≤ 512 MB (config default) 2.8

Packet loss under load iperf3+nstat -az TcpRetransSegs, UdpinErrors No new retransmit pattern

2.9 Sensor health sudo /opt/CrowdStrike/falconctl-g--status Operational: true, no RFM Must be fully operational

3. Post-Install with Network Visibility Module (Known Worst Case)

Test Action What to Measure Acceptable Delta

3.1 UDP send iperf3 c <peer> -u-b 10G -t 60 Gbps, loss% $\leq 15\%$ reduction (reported up to 50% on G9 flag if $> 20\%$)

3.2 TCP send iperf3c <peer> -t 60 Gbps $\leq 10\%$ from To

3.3 Latency ping/mtr P99 RTT ≤ 2 ms increase

3.4 Storage 1/0 fio IOPS, p99 $\leq 10\%$ from 10.

4. AI Discovery / Runtime Protection Specific Check

X

Telemetry volume delta Compare falconctl stats event count with/without AI Discovery policy
Events/min delta 4.1

I

Test Action What to Measure Expected Result

<10% increase (Discovery is

cloud-side analysis of existing telemetry)

mpstat during active AI agent session on the server Per-core delta vs. step 2.6 <1-2% additional
4.2 CPU delta

4.3 4.4 Memory delta free-m/ RSS of sensor during AI agent session MB delta vs. step 2.7 <10
MB additional Network egress to cloud tcpdump -i ethe host *.cloudsink.net or iftop KB/s to
CrowdStrike cloud

4.5 No new local computation strace -p \$(pidof falcon-sensor) c (brief) Steady, <100 KB/s
typical Syscall profile No new heavy syscalls vs. baseline

5. Service Readiness Gate (Go/No-Go)

Gate Criteria

G1: Compatibility

Owner Kernel version in CrowdStrike support matrix; NIC driver compatible; no conflicting
security agents

Platform / SRE

G2: Baseline captured All To, Uo, Le, 10. values recorded and signed off Network/Storage team

G3: Sensor-only test passed All deltas within thresholds in §2 Security / SRE

G4: §4 deltas within threshold Security / SRE

G5: AI modules test passed

Network Visibility decision Explicitly enabled or disabled; if enabled, §3 deltas within threshold
Security team

G6: Firewall rules verified

Outbound TCP 443 to cloudsink.net confirmed; no inbound rules needed

Network team

G7: Bandwidth policy set

Channel File Bandwidth Control policy configured if link <10 Mbps

Network team

GB: Update channel selected

n-2 (recommended for critical servers); maintenance window scheduled

SRE

G9: Rollback plan documented

falconctl stop command, sensor uninstall token, re-test procedure

SRE/ Security

G10: Monitoring in place

Falcon for IT ResourceUtilization per-process query configured; alert on sensor CPU >25% or RSS >512 MB SOC / SRE

6. Rollback & Emergency Procedures

Action Command / Step

Stop sensor (immediate) `sudo systemctl stop falcon-sensor`

Verify throughput restored Re-run iperf3, compare to To `sudo /opt/CrowdStrike/falconctl -g-uninstall` (requires maintenance token)

Uninstall sensor Re-test Full §1 baseline re-run

Document Log incident, sensor version, kernel, NIC driver, iperf results

Key References

B

Source Data Point

r/crowdstrike (2024) G9 Linux: 20% UDP/50% TCP send reduction (sensor); 50% UDP (Network Visibility) SmartInfralog deployment guide (2026) ~2.1% CPU under I/O-heavy load; 200-350 MB RAM steady

CPU limit 25%, CrowdStrike config defaults memory limit 512 MB, n-2 update channel for critical servers

CrowdStrike deployment FAQ

I

Scales to 100k endpoints with "no performance impact" (cloud-side)

Linux sensor guide -10-day latency between kernel release and CrowdStrike support do not upgrade kernel before confirming

Bottom line: For firewalls, load balancers, and storage nodes, the base sensor's kernel hooks are the risk, not AI Discovery. Run the iperf baseline sensor-only (optional) Network Visibility AI modules sequence, and gate on the thresholds above. If you hit the 69-level degradation (20-50%), the fix is to disable Network Visibility on those nodes and/or escalate to CrowdStrike support with the iperf results AI Discovery itself is not the culprit.

Copy

Does load balancer health check latency increase with sensors?

Checked for health check latency impacts from CrowdStrike

Short answer: under normal conditions, no measurable increase but there are specific scenarios where health check latency can degrade.

I

Why it's usually negligible

Health checks are short-lived (a TCP 3-way handshake or a single HTTP GET, completing in <1ms on the wire). The sensor's per-packet inspection (WFP) callouts on Windows, LSM/BPF hooks on Linux) adds microseconds of processing per packet. For a 1-10 second health check interval, that's invisible.

Where it can show up

Scenario

Mechanism

Expected impact on health check RTT

Normal operation

WFP classify / kernel hook per packet <0.1 ms undetectable

Health check RTT may rise from ~1 ms to 2-5

G9 Linux + sensor active (known 50% TCP reduction)

Kernel hook contention under high connection rate

ms; if LB timeout is tight (e.g., 2 s), still passes, but margin shrinks

G9+ Network Visibility enabled Stream DPI on every flow (FwpsCloneStreamData) Higher per-packet cost; RTT could rise further under sustained connection

10,000-flow cap reached (Windows)

churn New connections stop being tracked sensor skips classification no added cost, but also no visibility No

latency impact, but detection gap

Sensor in RFM/kernel panic (Sensor update in progress

Linux) Module fault, system instability Brief CPU spike during binary swap Transient 1-3 s latency spike on one check cycle

Health check fails entirely (timeout), not just slower

The real risk for load balancers

The load balancer itself is typically a high-connection-rate appliance (tens of thousands of concurrent flows). Two specific concerns:

Flow tracking saturation (Windows): The WFP engine caps at 10,000 tracked flows. A busy LB (HAProxy, Nginx, F5, AWS NLB targets) will exceed this quickly. Beyond the cap, new connections are untracked the sensor stops adding per-packet cost (good for latency) but also stops providing visibility (bad for security).

Connection churn amplifies hook cost (Linux): Each new connection triggers execve-adjacent hooks and flow-establishment callbacks. A LB creating/destroying thousands of connections per second means the sensor's kernel hooks fire at that rate. This is the mechanism behind the G9 throughput numbers. The per-connection overhead is small, but at LB scale it compounds.

Practical test for your LB

Step Command / Action

What to look for

1. Baseline health check RTT `curl -o/dev/null -s -w '%(time_total)\n' http://<backend>/health` (run 100x) Record median and p99

2. With sensor active, no AI modules Same command Delta in p99

sensor active p99 RTT under 1000 concurrent connections 3. Under simulated load `wrk -t8-c1000-d60s http://<backend>/health`

Compare to step 3 4. With Network Visibility (if applicable) Repeat step 3

5. Check flow count `netsh wfp show state` (Win) or `sss` (Linux) Confirm you're not near the 10K cap

Pass criteria: p99 health check RTT increase ≤ 1 ms under normal load; ≤ 5 ms under the G9 worst-case profile. If you exceed that, the fix is to exclude the LB process from Network Visibility or move the LB to a sensor-exempt group.

AI Discovery / Runtime Protection specifically

These modules add zero per-packet processing. They analyse the telemetry after the sensor has already captured it. For health check latency, the AI modules are irrelevant only the base sensor's network hooks and (if enabled) Network Visibility matter.

Bottom line: For a typical LB, expect no measurable health check latency increase from AI Discovery. The base sensor's kernel hooks add microsecond-level per-packet cost that only becomes visible under the G9-level degradation scenario or at extreme connection rates. Test with your actual health check interval and timeout to confirm.

Researched Falcon sensor impact on TCP handshake latency

Short answer: the sensor adds a few microseconds per handshake packet (LB, proxy, API gateway). invisible at normal connection rates, but measurable under high connection churn

What happens to a TCP handshake with the sensor active

Based on the reverse-engineered WFP callout architecture (from the @xdbgman teardown, Aug 2026), a single TCP 3-way handshake passes through multiple WFP layers where the sensor has registered callouts

Handshake packet

Sensor action

WFP layers hit (Windows)

SYN (outbound) OUTBOUND TRANSPORT V4 OUTBOUND_IPPACKET_V4 INBOUND IPPACKET_V4 INBOUND_TRANSPORT V4ALE_AUTH_RECV_ACCEPT_V4 Classify, extract 5-tuple, associate with Classify, match to process, check policy

SYN (inbound)

SYN-ACK (outbound) OUTBOUND_TRANSPORT_V4 Classify, update flow state

ALE_FLOW_ESTABLISHED_V4 CS_FlowEstablish Classify allocates the CSFR flow record, associates it as context for all future packets on this owning process (PID from ALE)

ACK (final)

connection

Each callout is a kernel-mode function call (classifyFn) that:

Reads packet metadata (IP, TCP headers)

Looks up or creates a flow record

Checks against the policy engine (cloud-loaded rules)

X

Returns FWPS CLASSIFY_CONTINUE (allow) or FWPS_CLASSIFY_DROP

Per-callout cost: -1-5 μ s (typical for a WFP callout doing a hash-table lookup policy check), with 3-4 callouts per handshake packet, the total added latency per handshake is roughly 5-20 μ s under normal conditions.

On Linux (LSM/eBPF backend)

The sensor loads falcon 1sm serviceable falcon 1sm_pinned kernel modules. A Reddit user (r/crowdstrike, 2024) confirmed via perf top:

"perf top reports a ton of methods like cshook reported as kernel methods. These are clearly CrowdStrike hook events. CrowdStrike does appear quite intrusive as it is sitting in the Linux kernel and these methods only appear when we kick up data rates."

The key phrase: "only appear when we kick up data rates." At low connection rates, the hook cost is amortised and invisible. At high rates, it shows up in the profile.

When it becomes measurable

Scenario Connections/sec Expected handshake latency delta

Detectable?

No (below RTT noise)

I

Typical workstation <10 <20 μ s No

Web server (moderate) 100-500 20-50 μ s

Load balancer / API gateway 1,000-10,000 50-200 μ s (compounded) Yes, with precise timing
G9 Linux worst case (50% TCP reduction) 10,000+ 200-500 μ s (queueing) Yes, clearly

The mechanism at high rates isn't that each individual callout gets slower it's that the kernel hook processing competes with the NIC's packet-processing pipeline, causing micro-queueing. This is the same mechanism behind the 20-50% throughput reduction: packets wait slightly longer in the kernel before being forwarded.

pipeline, causing micro-queueing. This is the same mechanism behind the 20-50% throughput reduction: packets wait slightly longer in the kernel before being

How to measure it precisely

#Baseline (sensor stopped)

sudo systemctl stop falcon-sensor

for i in \$(seq 1 1000); do

/usr/bin/time -f "%e" bash -c 'echo > /dev/tcp/<target>/443' 2>&1

```
done sort -n | awk (a[NR]=$1) END (print "p50: "a[500], "p99: "a[990])"
```

#With sensor active

```
sudo systemctl start falcon-sensor
```

#Repeat same loop

Or more precisely with a raw TCP client that times the SYN-ACK:

Using python (measures SYN-ACK RTT specifically)

```
python3.c"
```

```
import socket, time
```

```
for in range(1000):
```

```
    s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    s.settimeout(5)
```

```
    to=time.perf_counter()
```

```
    s.connect(('<target>', 443))
```

```
    titime=time.perf_counter()
```

```
    s.close()
```

```
    print(f'((t1-t0)*1e6;.1f) us')
```

```
"sort -n awk '{a [NR]=$1) END (print "p50: "a[500], "p99: "a[990])"
```

|

Pass criteria: p99 handshake latency increase $50\ \mu\text{s}$ at normal rates; $200\ \mu\text{s}$ at 10K+ conn/s. If you exceed that, the fix is the same as for the throughput issue exclude the LB process or disable Network Visibility on that node.

AI Discovery / Runtime Protection These modules add zero per-packet processing to the handshake path. They don't register additional WFP callouts or LSM hooks. The handshake latency is

determined entirely by the base sensor's network hooks and (if enabled) Network Visibility's stream DPI (FwpsCloneStreamData/FwpsCopyStreamDataToBuffere), which only kicks in after the handshake completes and data flows.

Bottom line: Under normal conditions, the sensor adds ~5-20 μ s to a TCP handshake well below any application-level timeout or user-perceivable threshold.

Bottom line: Under normal conditions, the sensor adds ~5-20 μ s to a TCP handshake well below any application-level timeout or user-perceivable threshold. It only becomes a concern at extreme connection rates (10K+ conn/s) where kernel-hook queueing compounds, and even then the fix is operational (exclude the process, disable Network Visibility), not architectural.

falcon.crowdstrike AI discovery and governance activation check list and prechecklist and impact pre-deployment impact analysis

AI Discovery & Endpoint Performance Impact

Short answer: minimal. AI Discovery (part of Falcon AIDR / Falcon for IT) operates on the existing Falcon sensor require separate tooling. it does not deploy an additional agent or

How it works resource-wise

AI Discovery analyses telemetry already being collected by the Falcon sensor in real-time, using:

Application fingerprinting

Package metadata heuristics

Command-line usage patterns

Process-to-process relationships

Marketplace integrations

This means no extra scan cycles, no additional disk I/O from local engines, and no new credential vaults or scan engines on the endpoint.

Baseline sensor footprint The Falcon sensor itself is a single lightweight agent with a footprint under 40 MB. Real-world user reports (from r/crowdstrike) put typical consumption at:

Resource

Typical usage

CPU 0-1% (occasionally up to 3%)

RAM

Disk

-10-15 MB

Minimal (config files/drivers only)

Caveats

H1

occasional cloud-service issues have historically caused degraded boot times and The main performance risk remains the Falcon sensor itself responsiveness, but that's platform-level, not AI-Discovery-specific.

X

If you enable AI Runtime Protection (which inspects live commands, scripts, file activity, and network connections from AI processes), there is slightly more telemetry volume, but CrowdStrike still positions this as operating within the existing sensor's resource envelope.

Bottom line: enabling AI Discovery on endpoints that already run the Falcon sensor adds no meaningful measurable overhead. The discovery logic is cloud-side analysis of data the sensor is already streaming.

Short answer: AI Discovery itself has no meaningful battery impact it's cloud-side analysis of telemetry the sensor is already streaming. Any battery drain users notice comes from the base Falcon sensor, not the AI Discovery module specifically.

What the data shows.

Source Reported impact

CrowdStrike / LBL FAQ $\leq 1\%$ CPU, "no perceptible performance impact" CrowdStrike staff (Reddit) 0-3% CPU, 10-15 MB RAM typical

HN users (2022, older sensor) 30-50% CPU in some cases (version-specific bug, fixed in 6.34)

The Stack (June 2024 bug) 100% of a single core (rolled back)

MacRumors forum User concerned about "constant scanning" draining battery

Apple App Store (mobile app) Complaints of heavy background battery drain I

Why the base sensor can affect battery

The sensor runs 24/7 in the background, continuously collecting and streaming telemetry.

On laptops, any persistent background process that wakes the CPU prevents deeper sleep states, incrementally shortening battery life. At the typical 8-3% CPU, the effect is small but measurable roughly equivalent to a few minutes of extra screen-on time per day.

Under heavy workloads (compilation, Docker, large file I/O), the sensor's process-monitoring hooks can spike temporarily, which is where users report fans spinning up and faster drain.

When it's more than negligible

Sensor bugs (e.g., the June 2024 memory-scanning bug at 100% of a core) can dramatically drain battery until patched.

Sensor bugs (e.g., the June 2024 memory-scanning bug at 100% of a core) can dramatically drain battery until patched.

Older sensor versions (6.33 and earlier on macOS) had known high-CPU issues.

Falcon Mobile (the 105/Android companion app) has separate, more visible battery complaints this is a different component from the endpoint sensor. AI Runtime Protection (if enabled alongside Discovery) inspects live commands, scripts, and network connections, adding slightly more telemetry volume still within the sensor's normal envelope, but marginally more than Discovery alone.

Practical guidance for laptop users

Keep the sensor updated most dramatic battery issues have been version-specific bugs, quickly patched.

Check per-process metrics via Falcon for IT (the new module lets you query ResourceUtilization per process to confirm the sensor isn't the culprit). If battery drain is noticeable, compare against other always-on agents (BigFix, MDM, DLP) in many cases the sensor is not the largest consumer.

AI Discovery alone won't change your battery life it doesn't add local computation.

Bottom line: the marginal battery cost of enabling AI Discovery on an already-deployed Falcon sensor is effectively zero. If you're seeing battery issues, they're from the base sensor (usually a bug or heavy workload), not from the Discovery module.

Checked CPU impact of AI Runtime Protection

Short answer: slightly more than Discovery alone, but still modest the main added cost is memory (maintaining agent context and causal chains) rather than

raw CPU.

What AI Runtime Protection adds on top of Discovery

Layer What it does Resource lcost

Discovery Fingerprinting AI apps/models via existing telemetry -Zero marginal (cloud-side analysis)

Runtime Protection Captures live commands, scripts, file I/O, and network connections specifically from AI processes and builds a causal chain (prompt tool callskill downstream action) Small additional telemetry volume in-memory context

identity Agent Access Controls (Falcon Guardian) Enforces allow/block policies at runtime Negligible (policy lookup)

Why CPU impact is limited

The Falcon sensor already monitors process creation, file access, command-line activity, and network connections for all applications. AI Runtime Protection doesn't add a new monitoring mechanism it correlates and tags existing telemetry events that originate from known AI processes. The causal-chain logic (linking a user prompt to a tool call to a file write) is largely cloud-side analysis, not endpoint-side computation. CrowdStrike's own positioning: the feature "operates within the existing sensor's resource envelope."

Where the marginal cost shows up Memory: One third-party analysis notes that "memory requirements increase slightly for endpoints running AI workloads, as the sensor needs to maintain

Memory: One

One third-party analysis notes that "memory requirements increase slightly for endpoints running AI workloads, as the sensor needs to maintain context about model states and data flows." This is the most measurable delta likely a few Me extra for the agent inventory and execution-graph state. CPU: The incremental CPU cost is in the range of the sensor's existing

script-based execution monitoring (5-8% in script-heavy environments per tuning guides). In practice, for most endpoints where AI agents are idle or lightly used, the delta is <1-2% CPU above the base sensor. NPU offload (future): CrowdStrike's collaboration with Intel demonstrated that AI inference models running endpoint-side can be offloaded to the NPU, dropping CPU usage from ~20% to <1% for comparable workloads. This isn't yet standard in production sensors but signals the direction.

Practical expectation.

Scenario

Expected CPU delta vs. base sensor

No AI agents running on endpoint 0% (no AI processes to track) <1%

One AI agent idle (e.g., Copilot open but not active)

Active agent executing commands/scripts 1-3% (overlaps with existing script-monitoring overhead)

the sensor's normal 15-25% range during active work) Heavy agent workload (multiple concurrent tool calls, file I/O) 3-5% (still within the

Bottom line

X

AI Runtime Protection adds slightly more telemetry volume than Discovery alone (because it tracks live behavior, not just inventory), but the CPU cost is marginal essentially the difference between "monitor all processes" pro and "monitor all processes and tag the AI ones with causal context." The more noticeable resource delta is a small increase in RAM for maintaining the agent execution graph. For most endpoints, users will not perceive any difference.

Copy

Researched memory overhead for long-running AI sessions in CrowdStrike Falcon

Short answer: no public benchmark data yet Falcon Guardian launched at Fal.Con on September 1, 2026, and CrowdStrike hasn't published per-session memory profiles for extended agent runs. However, the architecture gives strong signals about what to expect.

Why the endpoint-side cost stays bounded

The key architectural fact is that Falcon Guardian is cloud-native the sensor streams telemetry and the heavy lifting (causal-chain reconstruction, blast-radius analysis, execution-graph correlation) happens in CrowdStrike's cloud. On the endpoint, the sensor maintains: I

A local agent inventory (which AI agents are running, their identity, deployment status) A sliding window of recent AI-process events (process creation, file I/O, network calls, command-line args)

A short-lived context buffer linking the most recent prompt tool call downstream action

This is not an unbounded log. The sensor is designed to stream events as they occur and retain only what's needed for local enforcement (blocking unauthorized agents, containing a session in real time). The full execution graph for investigation is assembled server-side in the Falcon platform.

X

What this means for long sessions

Session duration

Endpoint memory delta (estimated)

Minutes (typical chat) ~2-5 MB above base sensor

Hours (agentic workflow) -5-10 MB (bounded by sliding window, not cumulative) Days (autonomous agent) Same -5-10 MB ceiling old events are flushed to cloud, not retained locally

The critical design principle: the endpoint does not accumulate state proportional to session length. Each event is tagged, streamed, and then eligible for local eviction once the cloud has acknowledged receipt and the local enforcement window has passed.

Where memory can grow (edge cases)

Large tool outputs held in process memory if an AI agent itself allocates large buffers (e.g., loading a 500 MB dataset into a Python process), that's the agent's memory, not the sensor's. The sensor just observes the process.

High event rate an agent making thousands of tool calls per second will generate a higher telemetry throughput, briefly increasing the sensor's internal queue. This self-corrects as events are flushed to the cloud.

Local policy enforcement state if Agent Access Controls are actively blocking/enforcing, the sensor may retain slightly more context to make real-time allow/deny decisions. Still bounded.

Practical monitoring

Once Guardian is in your environment, you can verify actual memory behavior via:

Falcon for IT ResourceUtilization per process (the CrowdStrike sensor process specifically)
Windows: Task Manager Details sort by Memory, filter for CSFalconService.exe

macOS: Activity Monitor Memory tab, look for falcon process.

If you see the sensor's RSS growing linearly over hours with an active AI agent, that would be a bug (report to CrowdStrike support). The expected behavior is a flat or sawtooth memory profile it climbs as the queue fills, then drops when a batch is flushed to the cloud.

Bottom line

For long-running AI sessions, the Falcon sensor's memory overhead is bounded and session-length-independent by design. You should see a steady-state increase of roughly 5-18 MB over the base sensor footprint, regardless of whether the agent runs for 10 minutes or 10 hours. The "memory grows with session length" problem is a cloud-side concern (storage and query cost), not an endpoint one.

Researched network impact of Falcon AI for remote branches.

Short answer: AI Discovery adds essentially zero additional network traffic. It analyses telemetry the sensor is already streaming. The real network considerations for branch/remote sites are about the base sensor's telemetry stream, not the AI modules specifically.

Base sensor network profile

Factor Detail

Traffic type Outbound HTTPS (TLS) to cloudsink.net endpoints only

Volume Low and steady event-based, not bulk. Typical sensor sends a few KB/min under normal activity

Updates Monthly sensor binary update (tens of MB, one-time transfer)

Sensor caches events locally and syncs when connectivity returns no data loss, no re-scan

Offline behaviour Install requirement 10-minute connectivity window at first install

What AI Discovery / Runtime Protection adds to the wire

Discovery: -Zero. It fingerprints AI apps from metadata already in the telemetry stream. No new data is generated.

Runtime Protection: Marginally more events (command-line args, script content, file paths, network connection tuples from AI processes specifically). In practice this is a small fraction of total event volume likely <5-10% more telemetry bytes on endpoints with active AI agents, and -8% on endpoints without any.

Remote / branch office considerations

The real risk is the link, not the AI modules:

Link type Expected impact Mitigation

Broadband / fibre (210 Mbps). None perceptible

DSL/4G (1-10 Mbps) Negligible

VSAT/satellite (~1-2 Mbps, high latency) Sensor telemetry can compete with user traffic during bursts to cap upload rate; schedule sensor updates off-peak

Use "Channel File Bandwidth Control" policy
VSAT/ satellite (~1-2 Mbps, high latency)

Sensor telemetry can compete with user traffic during bursts.

to cap upload rate; schedule sensor updates off-peak

Intermittent/dial-up Sensor works offline, syncs on reconnect but detection latency increases

Acceptable for most use cases; document the gap

Known issue to be aware of: A Reddit user (2024) reported that on Linux servers with heavy network throughput, the Falcon sensor caused ~20% UDP and ~50% TCP send throughput reduction (iperf-tested). Enabling the Network Visibility module made it worse. This is a sensor-level kernel-hook cost, not AI-Discovery-specific, but it's the most concrete "network impact" data point in the wild. If your branch runs high-throughput Linux servers (firewalls, load balancers, storage), test with iperf before enabling all modules.

Practical guidance for branch deployments

Firewall rules: Allow outbound HTTPS (443) to CrowdStrike cloud domains (*.cloudsink.net, .crowdstrike.com). No inbound rules needed. Throttling: For links below 2 Mbps, set a Channel File Bandwidth Control policy (e.g., cap at 512 KB/s upload) to prevent telemetry

bursts from starving

user traffic. Update scheduling: Use the n-2 update channel and schedule sensor binary updates during off-peak hours (the one-time tens-of-MB download is the only significant one-time transfer).

No local analysis dependency: Because all detection logic runs in the cloud, branch sites don't need a local management server or large rule sets - the sensor just needs to reach the cloud. AI modules don't change any of this: If the base sensor is acceptable on your branch link, enabling AI Discovery or Runtime Protection won't change the

picture in any measurable way. network-invisible. For very low-bandwidth or Bottom line: For a typical branch office with 210 Mbps connectivity, AI Discovery and Runtime Protection are net high-latency links (satellite, rural), the base sensor's telemetry stream is the thing to manage use bandwidth throttling policies, not AI-module-specific tuning.

Copy

can you make a table and provide all information related to "firewalls, load balancers, storage), test with iperf" for pre testing service readiness

related to AI Discovery

Found pre-testing checklist for iperf on high-throughput Linux servers Pre -Testing & Service Readiness: High-Throughput Linux Servers

Based on the known data point (G9 Linux servers showing ~20% UDP send reduction and -50% TCP send reduction with the sensor active, worsened by Network Visibility), here's the full pre-testing and service readiness table for firewalls, load balancers, and storage nodes before enabling AI Discovery (or any

new module) at scale.

1. Pre-Deployment Baseline (Before Sensor Install)

Baseline Gbps jitter loss % Record as Uo

Test Tool / Command What to Record Pass Criteria

1.1 1.2 TCP throughput (send) iperf3 c <peer> -t 60 Baseline Gbps UDP throughput (send) iperf3c <peer> -u-b 10G -t 60 Record as To

1.3

TCP throughput (receive) iperf3s on peer, iperf3c <peer> -t 60 Baseline Gbps Record as To X

1.4 Latency (RTT) ping - 100 <peer> / mtr Avg, min, max, loss % Record as Lo 1/0 throughput (storage only) fio-rw-randrw-bs-4k-iodepth=32

1.5

IOPS, MB/s, latency p99 Record as 10.

1.6 CPU idle load top, mpstat -P ALL 1 Per-core utilisation, load avg Record as Co

1.7 Memory baseline free-m, cat /proc/meminfo Available, buffers, cached Record as Mo

1.8 Kernel version & driver stack uname-r, lsmod, ethtool -i ethe

Kernel, NIC driver, offload flags

defender

Must be in CrowdStrike compatibility Conflicting agents.

matrix

1.9 Existing security agents

systemctl list-units--type=service grep -E 'falcon ossec wazuh None, or documented exclusion

1.10 NIC offload status ethtool -k ethe GRO, GSO, TSO, LRO on/off

Document; sensor kernel hooks can interfere

2. Post-Install Sensor-Only Test (No AI Modules)

Action What to Measure Acceptable Delta vs. Baseline # Test

2.1 TCP send

iperf3 c <peer> -t 60 (sensor running) Gbps $\leq 5\%$ reduction from To iperf3 c <peer> -u-b 10G -t 60 Gbps, loss % $\leq 10\%$ reduction from U.

2.2 UDP send

2.3 TCP receive Same as 1.3 Gbps $\leq 5\%$ from Torx

2.4 Latency ping -c 100 ≤ 1 ms increase from Lo Avg RTT, p99

2.5 I/O (storage) fio same params as 1.5 IOPS, MB/s, p99 latency $\leq 5\%$ from 10.

2.6 CPU mpstat during iperf run Per-core peak $\leq 25\%$ of one core (per CrowdStrike config default)

2.7 Memory free -m during iperf run

RSS of falcon-sensor ≤ 512 MB (config default)

TcpRetransSegs, UdpInErrors 2.9 Sensor health sudo /opt/CrowdStrike/falconctl -g--status
Operational: true, no RFM Must be fully operational

No new retransmit pattern

3. Post-Install with Network Visibility Module (Known Worst Case)

I

Test Action What to Measure Acceptable Delta

3.1 iperf3c <peer> -u-b 106 -t 60 Gbps, loss% $\leq 15\%$ reduction (reported up to 50% on G9 flag if $> 20\%$)

UDP send

3.2 iperf3 c <peer> t 60 Gbps $\leq 10\%$ from To

TCP send

Latency ping/mtr p99 RTT ≤ 2 ms increase

3.3 3.4

Storage I/O fio